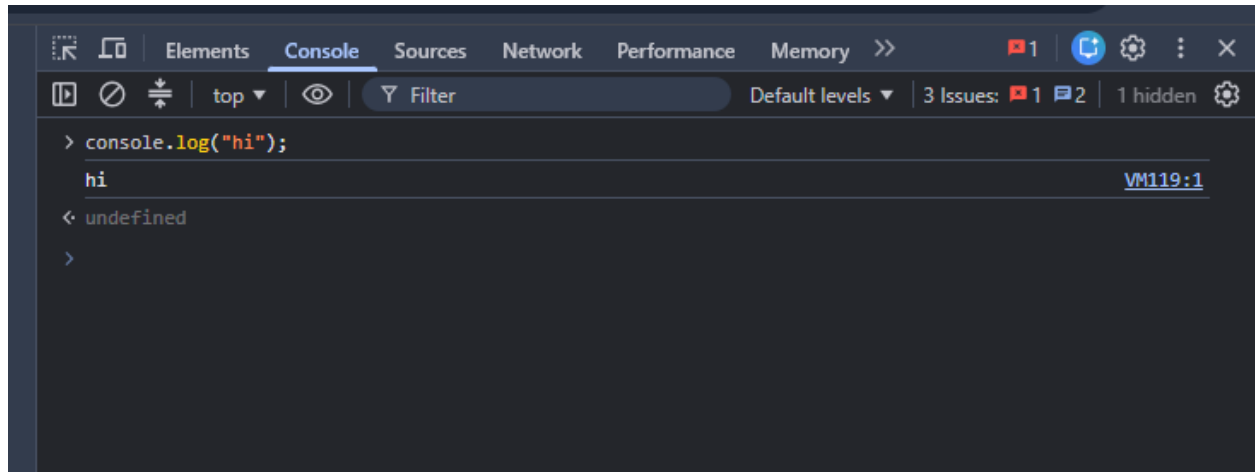


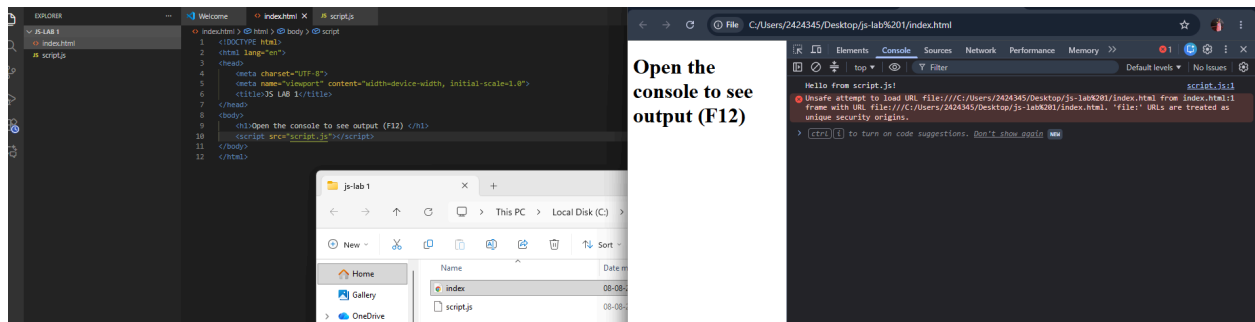
Tab 1

# Javascript lab assignment

## Method -1:



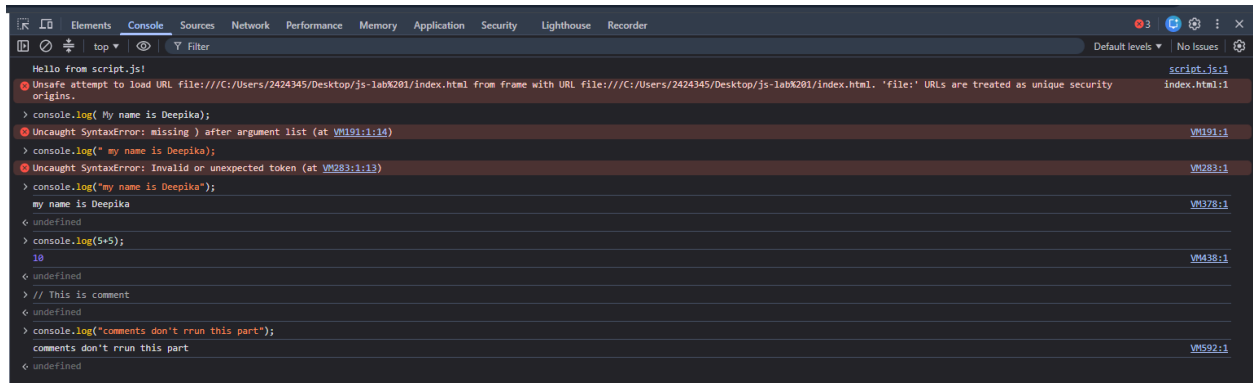
## Method -2:



Tab 2

# Part 1 — Syntax Warm-Up (10 min)

## Task 1.1:

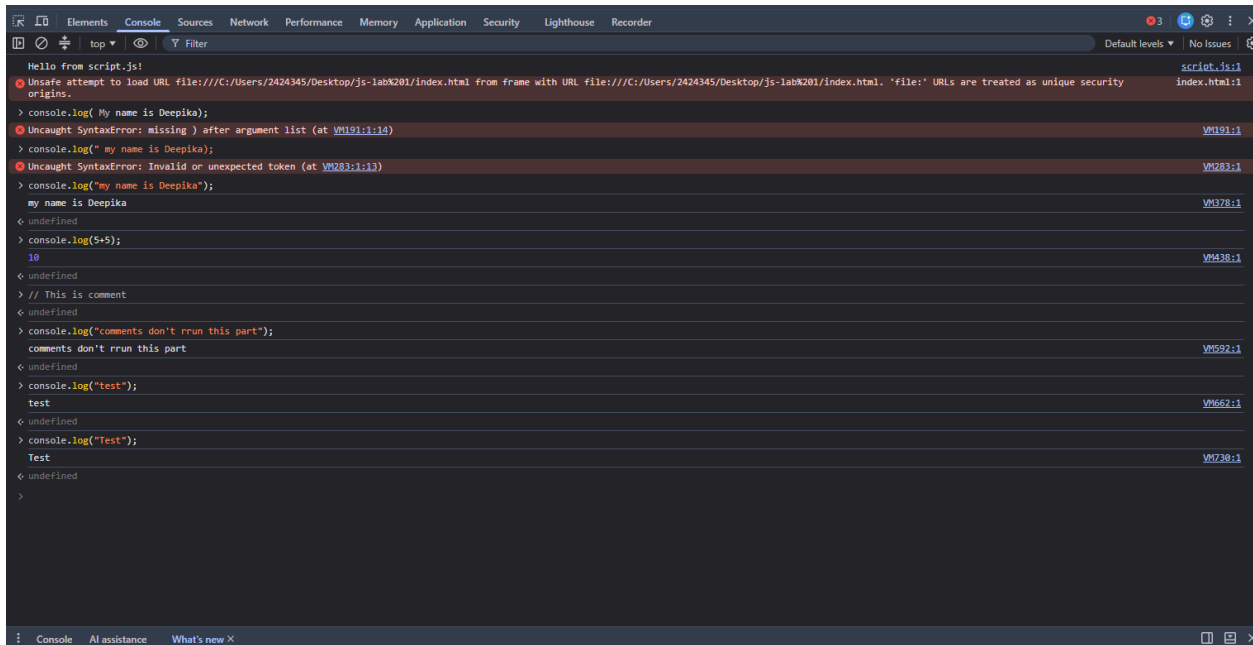


```

Hello from script.js!
Uncaught SecurityError: Unsafe attempt to load URL file:///C:/Users/2424345/Desktop/js-lab%201/index.html from frame with URL file:///C:/Users/2424345/Desktop/js-lab%201/index.html. "file:" URLs are treated as unique security origins.
    at index.html:1
    at script.js:1
> console.log( My name is Deepika);
Uncaught SyntaxError: missing ) after argument list (at VM191:1:14)
    at VM191:1:14
> console.log(" my name is Deepika");
Uncaught SyntaxError: Invalid or unexpected token (at VM283:1:13)
    at VM283:1:13
> console.log("my name is Deepika");
    my name is Deepika
    VM378:1
< undefined
    VM338:1
> console.log(5+5);
    10
    VM338:1
< undefined
    VM338:1
> // This is comment
    VM338:1
< undefined
    VM338:1
> console.log("comments don't rrun this part");
    comments don't rrun this part
    VM592:1
< undefined
    VM592:1
>

```

## Task 1.2:



```

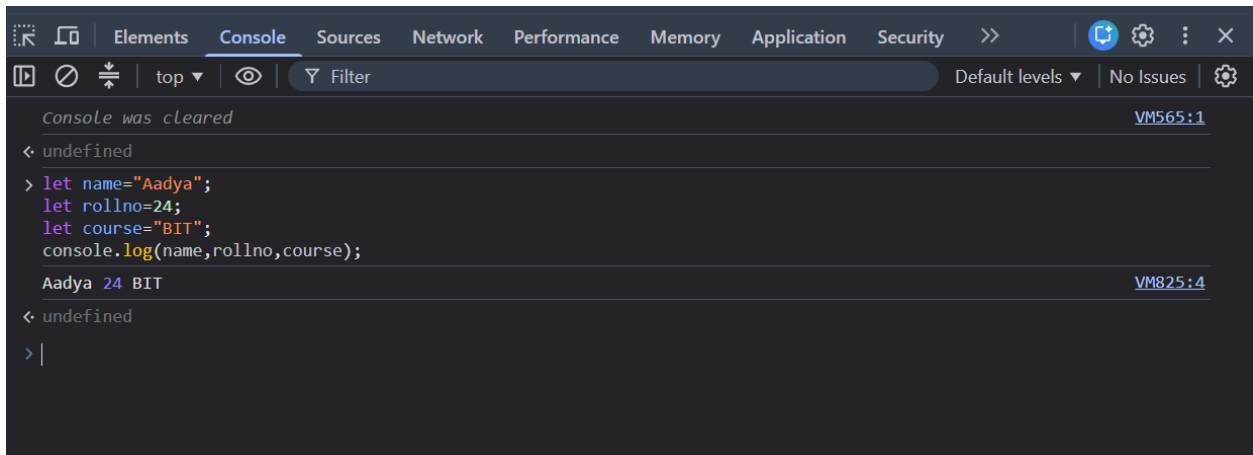
Hello from script.js!
Uncaught SecurityError: Unsafe attempt to load URL file:///C:/Users/2424345/Desktop/js-lab%201/index.html from frame with URL file:///C:/Users/2424345/Desktop/js-lab%201/index.html. "file:" URLs are treated as unique security origins.
    at index.html:1
    at script.js:1
> console.log( My name is Deepika);
Uncaught SyntaxError: missing ) after argument list (at VM191:1:14)
    at VM191:1:14
> console.log(" my name is Deepika");
Uncaught SyntaxError: Invalid or unexpected token (at VM283:1:13)
    at VM283:1:13
> console.log("my name is Deepika");
    my name is Deepika
    VM378:1
< undefined
    VM338:1
> console.log(5+5);
    10
    VM338:1
< undefined
    VM338:1
> // This is comment
    VM338:1
< undefined
    VM338:1
> console.log("comments don't rrun this part");
    comments don't rrun this part
    VM592:1
< undefined
    VM592:1
> console.log("test");
    test
    VM662:1
< undefined
    VM662:1
> console.log("Test");
    Test
    VM730:1
< undefined
    VM730:1
>

```

Tab 3

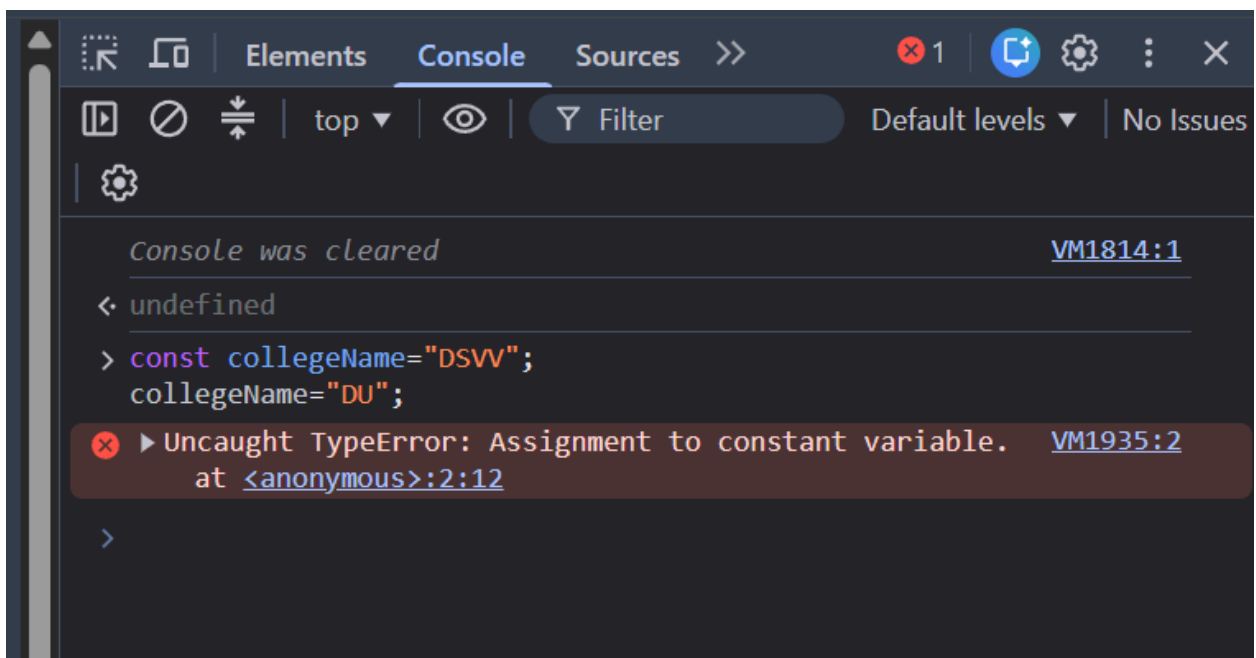
## Part -2: variables

### Task 2.1:



The screenshot shows the Chrome DevTools Console with the 'Console' tab selected. The console output shows a message 'Console was cleared' followed by 'undefined'. Then, a JavaScript code snippet is executed: `let name="Aadya"; let rollno=24; let course="BIT"; console.log(name,rollno,course);`. The output of the `console.log` statement is 'Aadya 24 BIT'. The console also shows 'undefined' and a prompt character `>`.

### Task 2.2:Reassign value



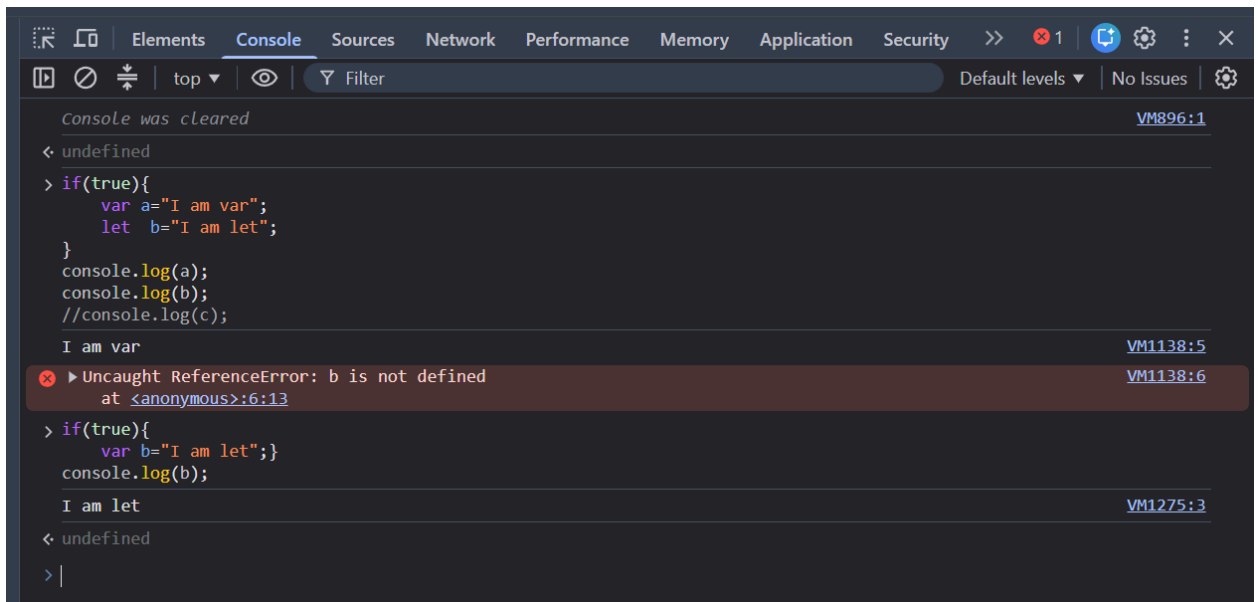
The screenshot shows the Chrome DevTools Console with the 'Console' tab selected. The console output shows a message 'Console was cleared' followed by 'undefined'. Then, a JavaScript code snippet is executed: `const collegeName="DSVV"; collegeName="DU";`. The output of the first line is 'DSVV', but the second line causes an error. The error message is 'Uncaught TypeError: Assignment to constant variable.' at `<anonymous>:2:12`. The console also shows 'undefined' and a prompt character `>`.

Reason- const value cannot be reassigned with new value ,so here is showing TypeError .

## Task 2.3:Naming Practice

- 1) let 1stYear; invalid . Reason- variable cannot start with a number.
- 2) let student\_name; valid. Reason- you can start with \_ or \$.
- 3) let let; invalid. Reason- let is a keyword.
- 4) let \$marks; valid. Reason- can start with \$.
- 5) let roll no; invalid. Reason- white spaces are not allowed .

## Task 2.4:



```

> if(true){
  var a="I am var";
  let b="I am let";
}
console.log(a);
console.log(b);
//console.log(c);
I am var
Uncaught ReferenceError: b is not defined
    at <anonymous>:6:13
> if(true){
  var b="I am let";
}
console.log(b);
I am let
> |
```

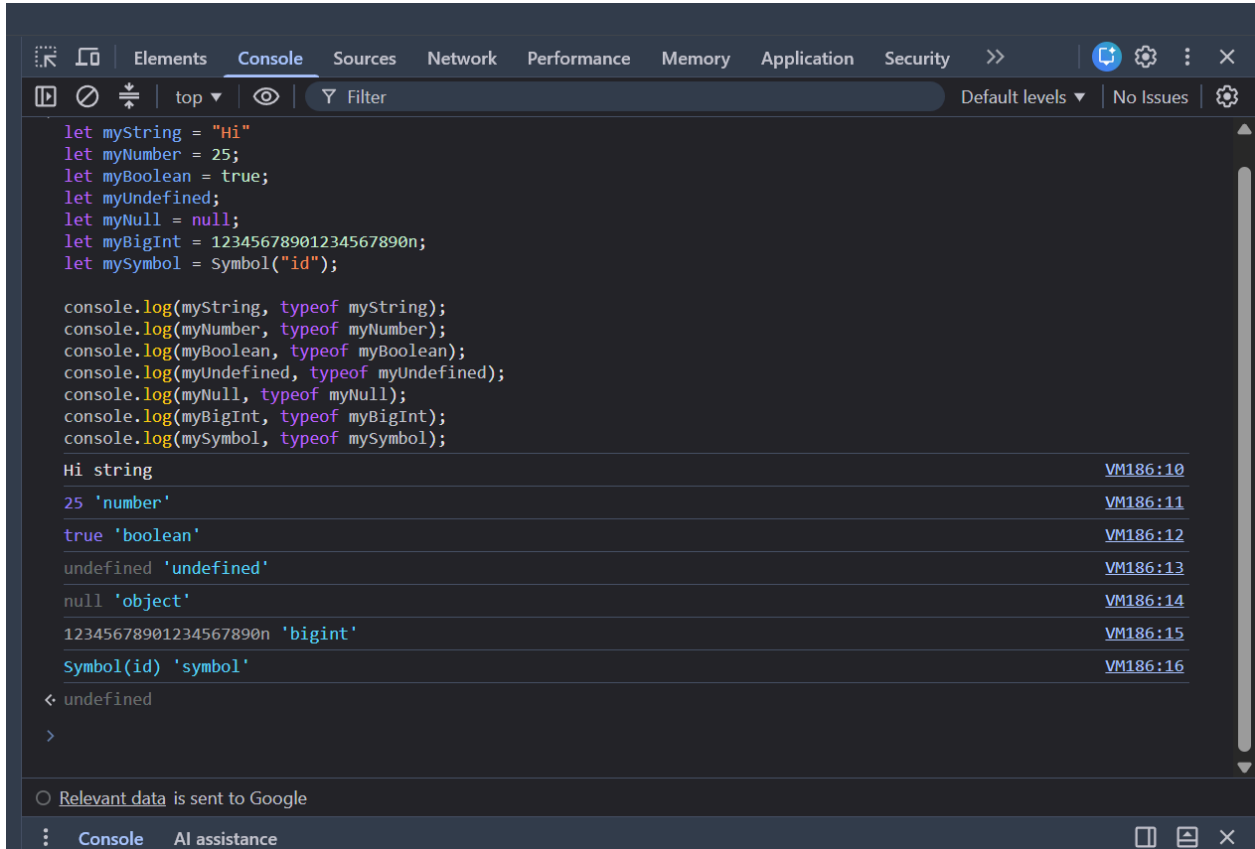
Reason- var is function-scoped, so the variable a can be accessed outside the if block. let is block-scoped, so variable b exists only inside the if block. Therefore, console.log(a) works, but console.log(b) gives a ReferenceError.

Tab 4



## Part -3 :Data Types

### Task 3.1



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following JavaScript code and its output:

```
let myString = "Hi";
let myNumber = 25;
let myBoolean = true;
let myUndefined;
let myNull = null;
let myBigInt = 12345678901234567890n;
let mySymbol = Symbol("id");

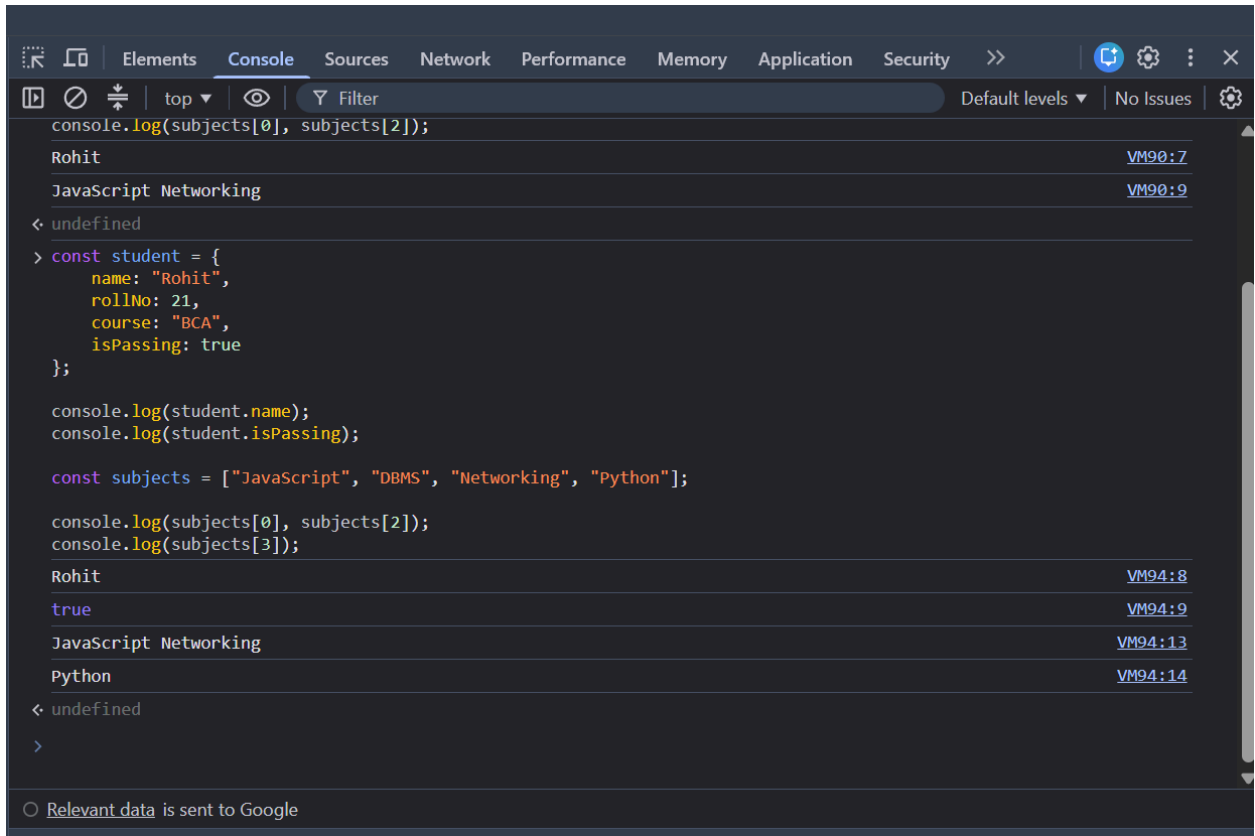
console.log(myString, typeof myString);
console.log(myNumber, typeof myNumber);
console.log(myBoolean, typeof myBoolean);
console.log(myUndefined, typeof myUndefined);
console.log(myNull, typeof myNull);
console.log(myBigInt, typeof myBigInt);
console.log(mySymbol, typeof mySymbol);
```

The output of the code is as follows:

|                       |             |          |
|-----------------------|-------------|----------|
| Hi                    | string      | VM186:10 |
| 25                    | 'number'    | VM186:11 |
| true                  | 'boolean'   | VM186:12 |
| undefined             | 'undefined' | VM186:13 |
| null                  | 'object'    | VM186:14 |
| 12345678901234567890n | 'bigint'    | VM186:15 |
| Symbol(id)            | 'symbol'    | VM186:16 |

Below the output, the console shows a prompt with the text "undefined" and a cursor. At the bottom of the console, there is a message: "Relevant data is sent to Google".

### Task 3.2 -object and array



The screenshot shows a web browser's developer console with the following code and output:

```
console.log(subjects[0], subjects[2]);
```

Output: Rohit (VM90:7), JavaScript Networking (VM90:9)

```
< undefined
```

```
> const student = {  
  name: "Rohit",  
  rollNo: 21,  
  course: "BCA",  
  isPassing: true  
};
```

```
console.log(student.name);  
console.log(student.isPassing);
```

```
const subjects = ["JavaScript", "DBMS", "Networking", "Python"];
```

```
console.log(subjects[0], subjects[2]);  
console.log(subjects[3]);
```

Output: Rohit (VM94:8), true (VM94:9), JavaScript Networking (VM94:13), Python (VM94:14)

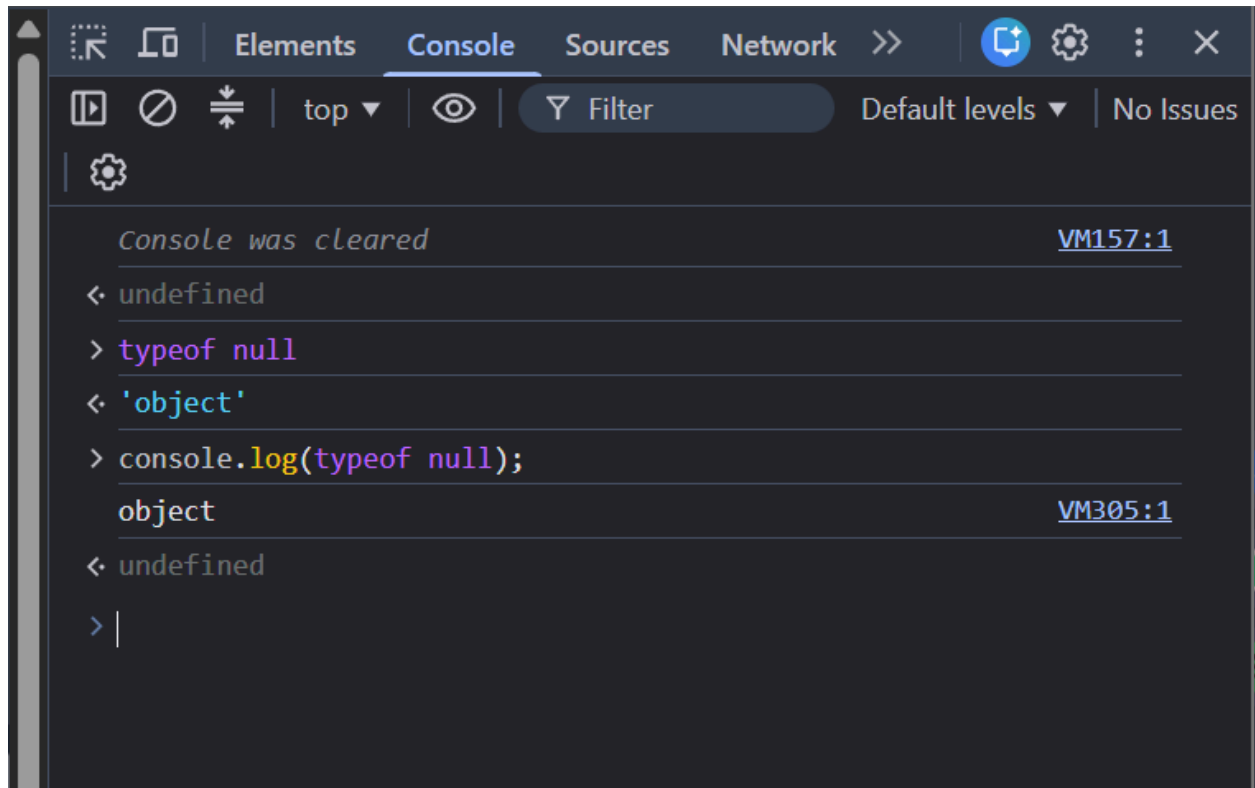
```
< undefined
```

```
>
```

At the bottom, there is a checkbox labeled "Relevant data" with the text "is sent to Google" next to it.

Added a 4th subject (python) whose index is 3 ,as it starts from 0.

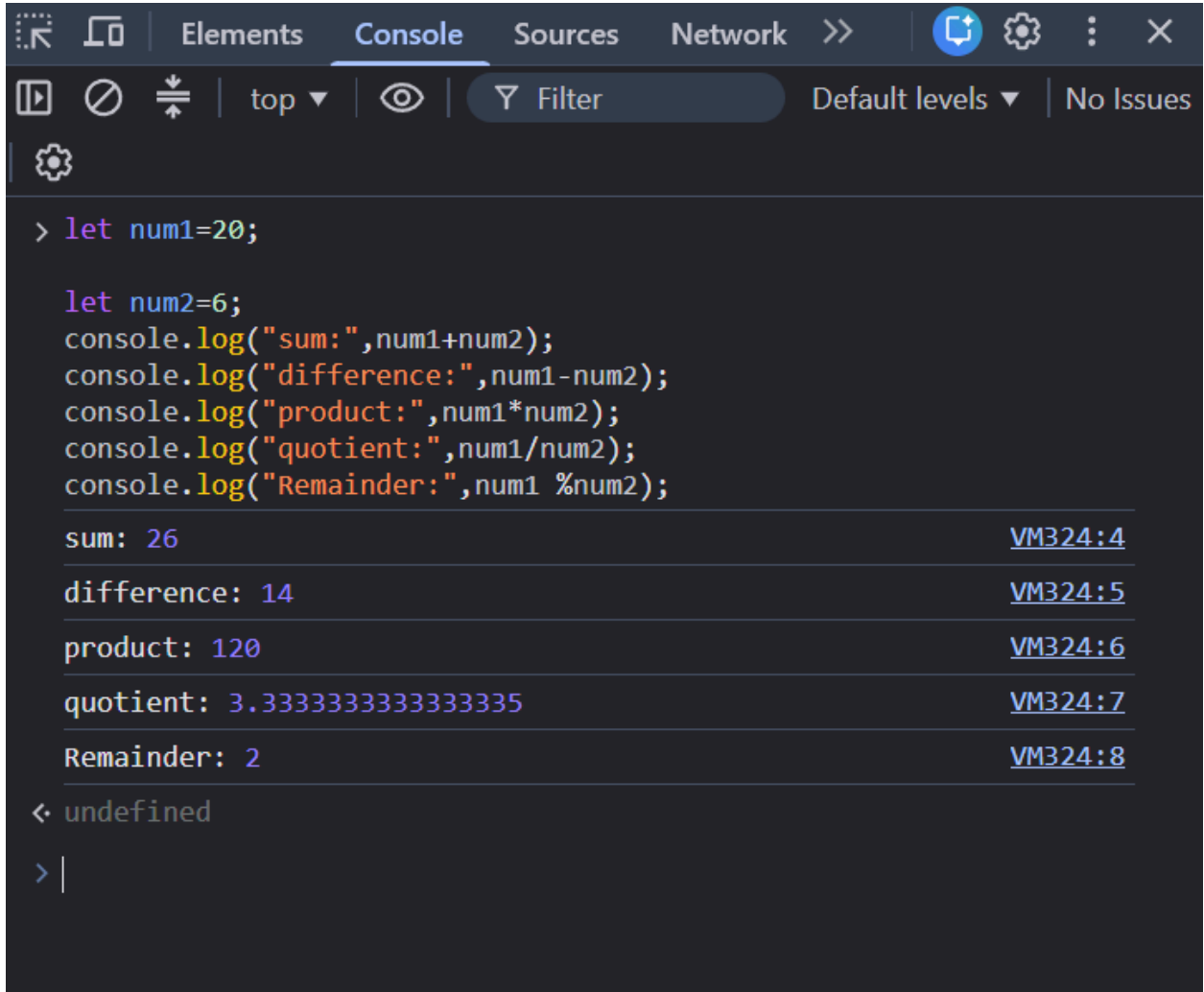
### Task 3.3: The typeof null Trick



Explanation: `typeof null` returns "object" because of a historical JavaScript bug/legacy behavior that has been kept for backward compatibility.

## Part -4: Operators and Expressions

### Task 4.1:



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following JavaScript code and its output:

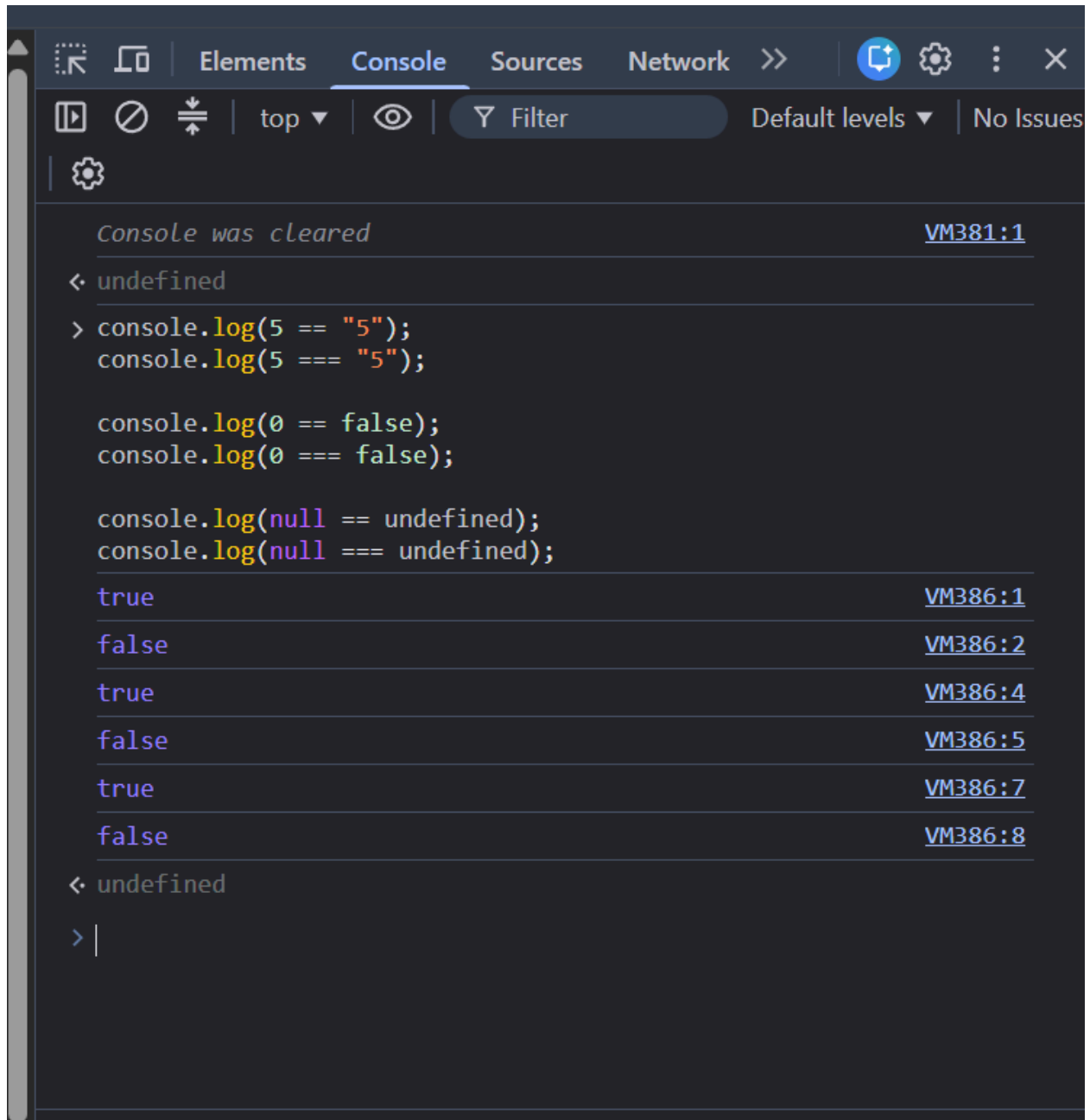
```
> let num1=20;

let num2=6;
console.log("sum:",num1+num2);
console.log("difference:",num1-num2);
console.log("product:",num1*num2);
console.log("quotient:",num1/num2);
console.log("Remainder:",num1 %num2);
```

|                              |                         |
|------------------------------|-------------------------|
| sum: 26                      | <a href="#">VM324:4</a> |
| difference: 14               | <a href="#">VM324:5</a> |
| product: 120                 | <a href="#">VM324:6</a> |
| quotient: 3.3333333333333335 | <a href="#">VM324:7</a> |
| Remainder: 2                 | <a href="#">VM324:8</a> |

Below the output, the console shows '< undefined' and a prompt '> |'.

### Task 4.2: == vs ===



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following sequence of events:

- A message: *Console was cleared* (source: [VM381:1](#))
- A prompt: `< undefined`
- Code input:

```
> console.log(5 == "5");
    console.log(5 === "5");

    console.log(0 == false);
    console.log(0 === false);

    console.log(null == undefined);
    console.log(null === undefined);
```
- Output lines (each with a source link):
  - `true` ([VM386:1](#))
  - `false` ([VM386:2](#))
  - `true` ([VM386:4](#))
  - `false` ([VM386:5](#))
  - `true` ([VM386:7](#))
  - `false` ([VM386:8](#))
- A prompt: `< undefined`
- A prompt: `> |`

### Explanation:

`5=="5"` - true. Reason- `==` converts "5" to number 5

`5==="5"`-false. Reason- number & boolean are different types

`0==false`-true.Reason- false is converted to 0

`0===false`- false.Reason-Number and Boolean are different types

`null==undefined` -true.Reason- `==` treats them as loosely equal

`null===undefined`-false.Reason- They are different types

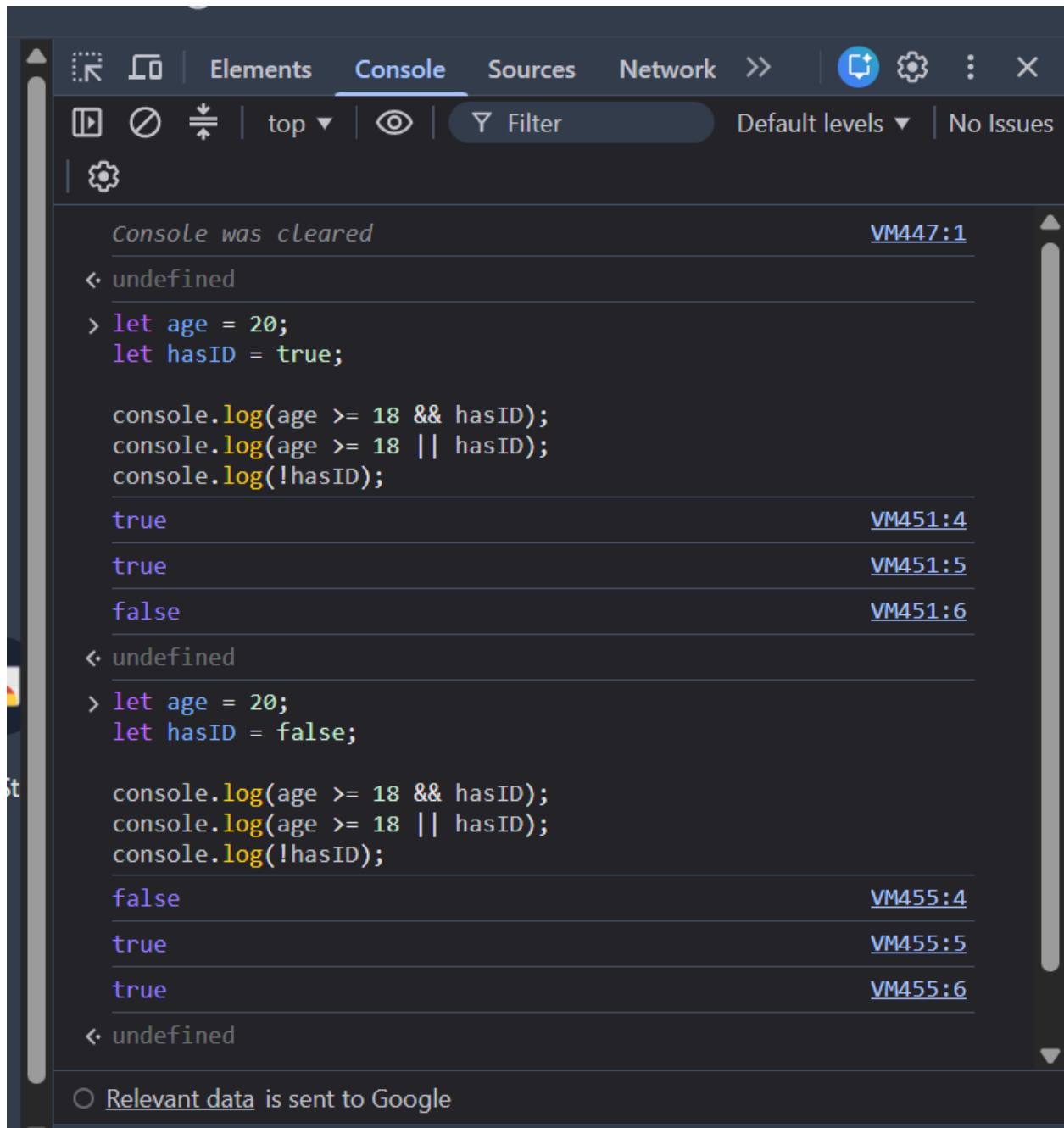
1)== → value comparison + type conversion 2)=== → value + type comparison

== checks equality after allowing type conversion, while === checks both value and data type without type conversion.

Therefore, === is stricter than ==.

Tab 5

## Part 4.3 — Logical Operators



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following sequence of events:

- A message: *Console was cleared* (source: VM447:1).
- A prompt: `< undefined`.
- Code execution:

```
> let age = 20;
let hasID = true;

console.log(age >= 18 && hasID);
console.log(age >= 18 || hasID);
console.log(!hasID);
```
- Output: `true` (source: VM451:4).
- Output: `true` (source: VM451:5).
- Output: `false` (source: VM451:6).
- A prompt: `< undefined`.
- Code execution:

```
> let age = 20;
let hasID = false;

console.log(age >= 18 && hasID);
console.log(age >= 18 || hasID);
console.log(!hasID);
```
- Output: `false` (source: VM455:4).
- Output: `true` (source: VM455:5).
- Output: `true` (source: VM455:6).
- A prompt: `< undefined`.

At the bottom of the console, there is a checkbox labeled `Relevant data` which is currently unchecked, followed by the text `is sent to Google`.

age=20,hasID=false

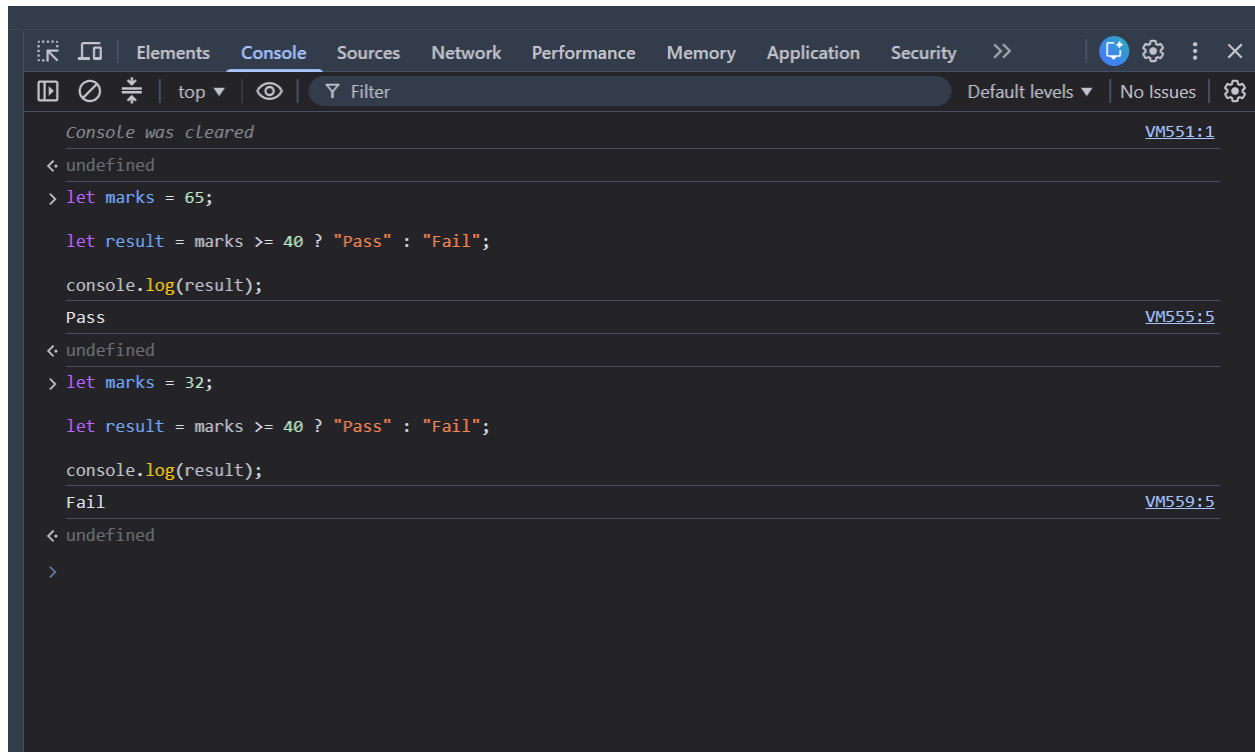
Reason: age >= 18 → true

hasID → false

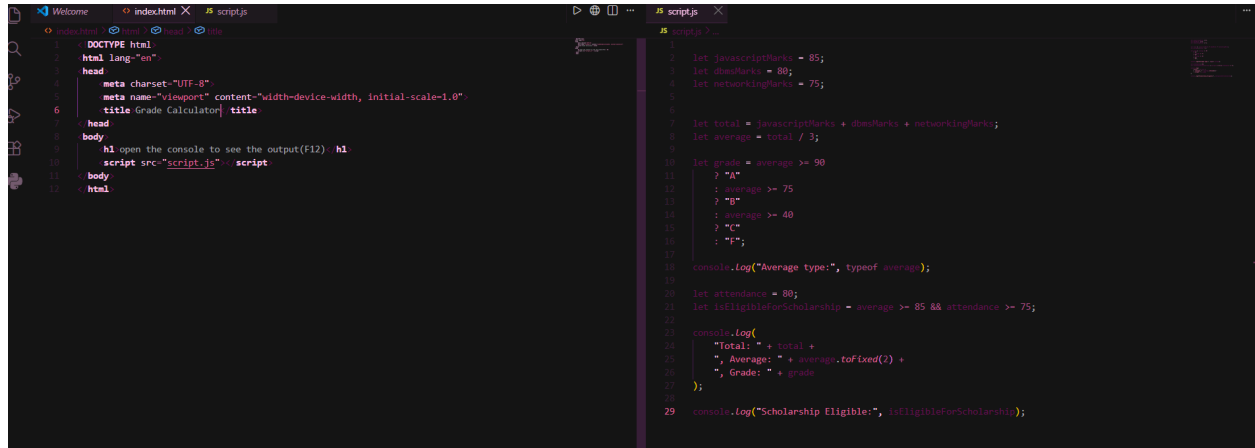
true && false → false

## Part 4.4 — Ternary Operator





## Part 5 — Simple Grade Calculator

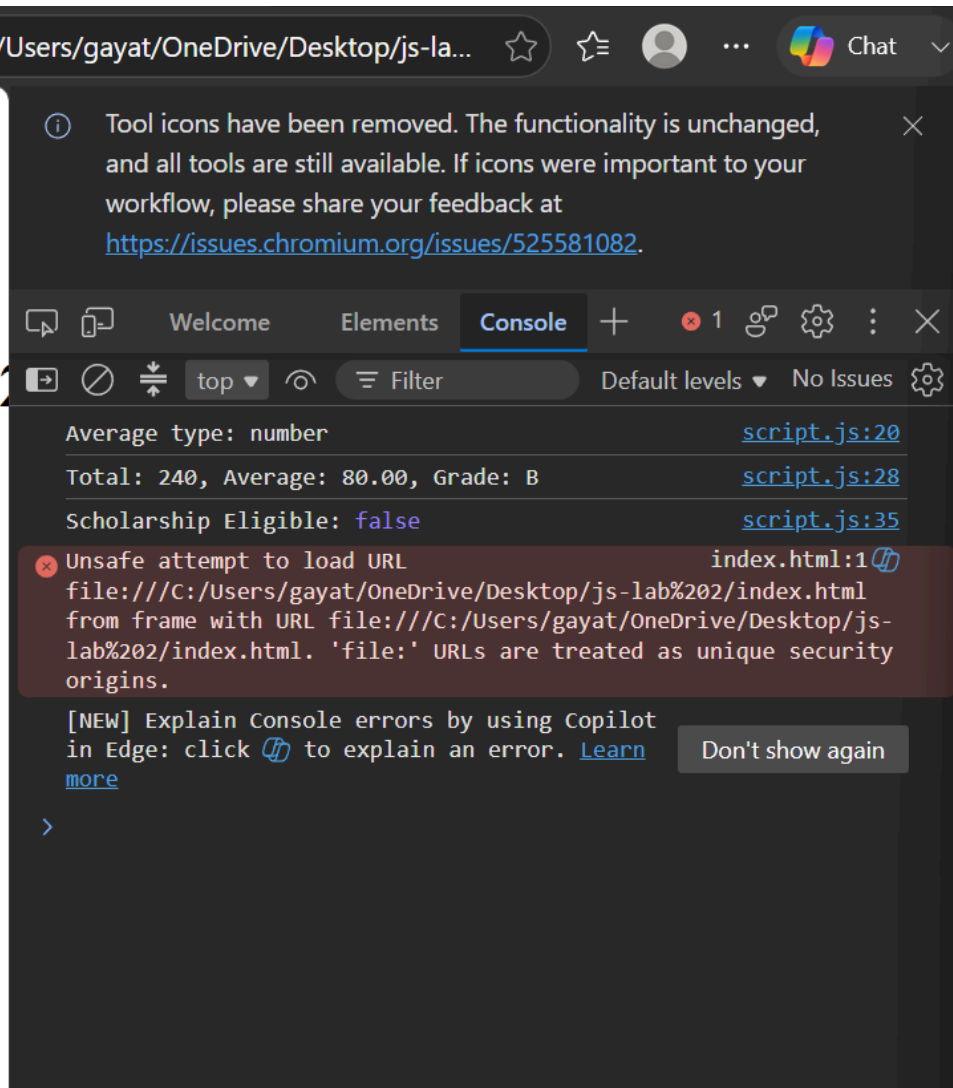


The image shows a code editor with two files: `index.html` and `script.js`. The `index.html` file contains a basic HTML structure with a title "Grade Calculator" and a script tag linking to `script.js`. The `script.js` file contains JavaScript code that calculates a grade based on three marks (javascriptMarks, dbmsMarks, and networkingMarks), calculates the average, and determines if the student is eligible for a scholarship based on the average and attendance.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title Grade Calculator</title>
7 </head>
8 <body>
9   <h1 open the console to see the output(F12)> h1
10   <script src="script.js"> </script>
11 </body>
12 </html>
```

```
1
2 let javascriptMarks = 85;
3 let dbmsMarks = 80;
4 let networkingMarks = 75;
5
6
7 let total = javascriptMarks + dbmsMarks + networkingMarks;
8 let average = total / 3;
9
10 let grade = average >= 90
11   ? "A"
12   : average >= 75
13   ? "B"
14   : average >= 40
15   ? "C"
16   : "F";
17
18 console.log("Average type:", typeof average);
19
20 let attendance = 80;
21 let isEligibleForScholarship = average >= 85 && attendance >= 75;
22
23 console.log(
24   "Total: " + total +
25   ", Average: " + average.toFixed(2) +
26   ", Grade: " + grade
27 );
28
29 console.log("Scholarship Eligible:", isEligibleForScholarship);
```

**open the  
console  
to see the  
output(F12)**



## Why is scholarship false?

The condition is:

`average >= 85 && attendance >= 75`

Your values are:

`average = 80`

`attendance = 80`

So:

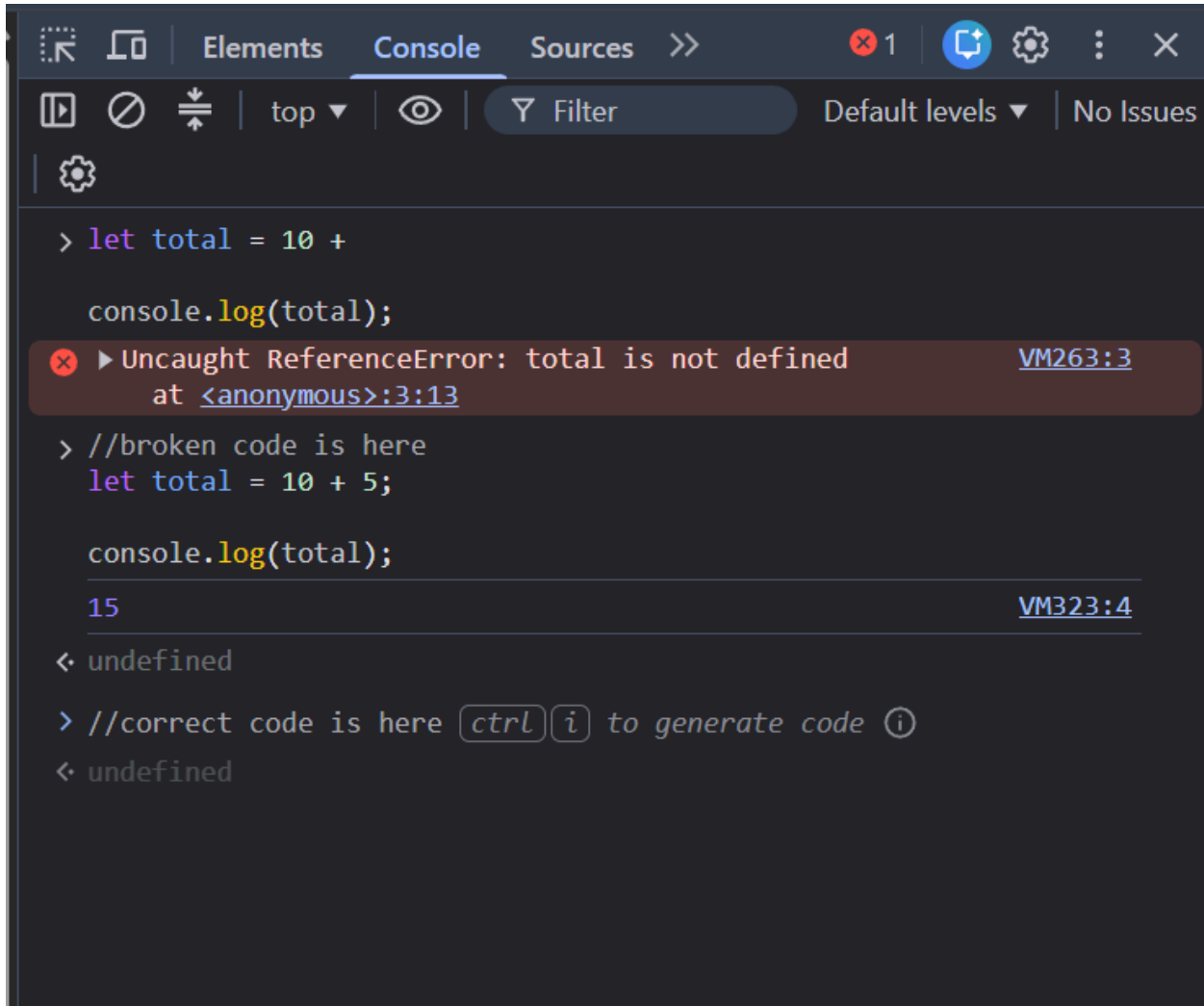
`80 >= 85 → false`

`80 >= 75 → true`

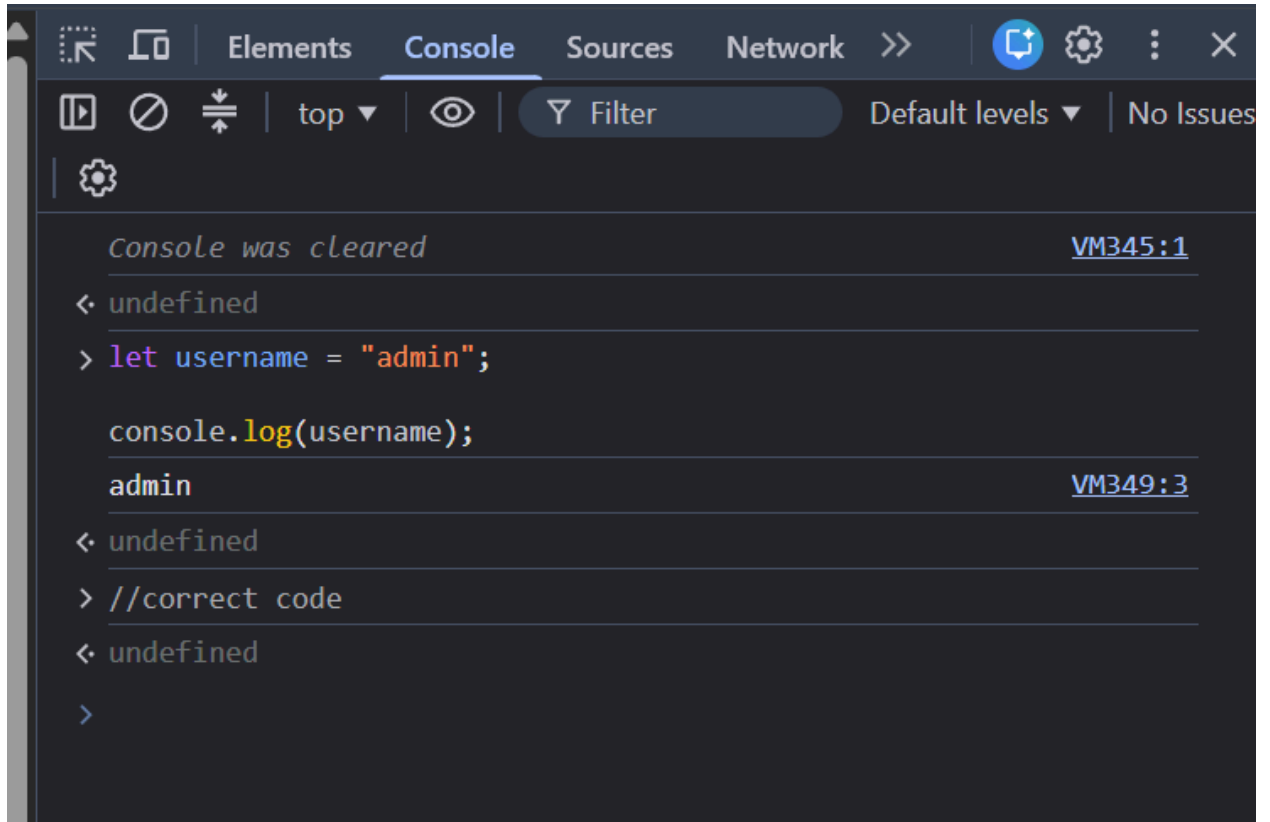
false && true → false

## Part 6 — Debugging

Snippet-1:



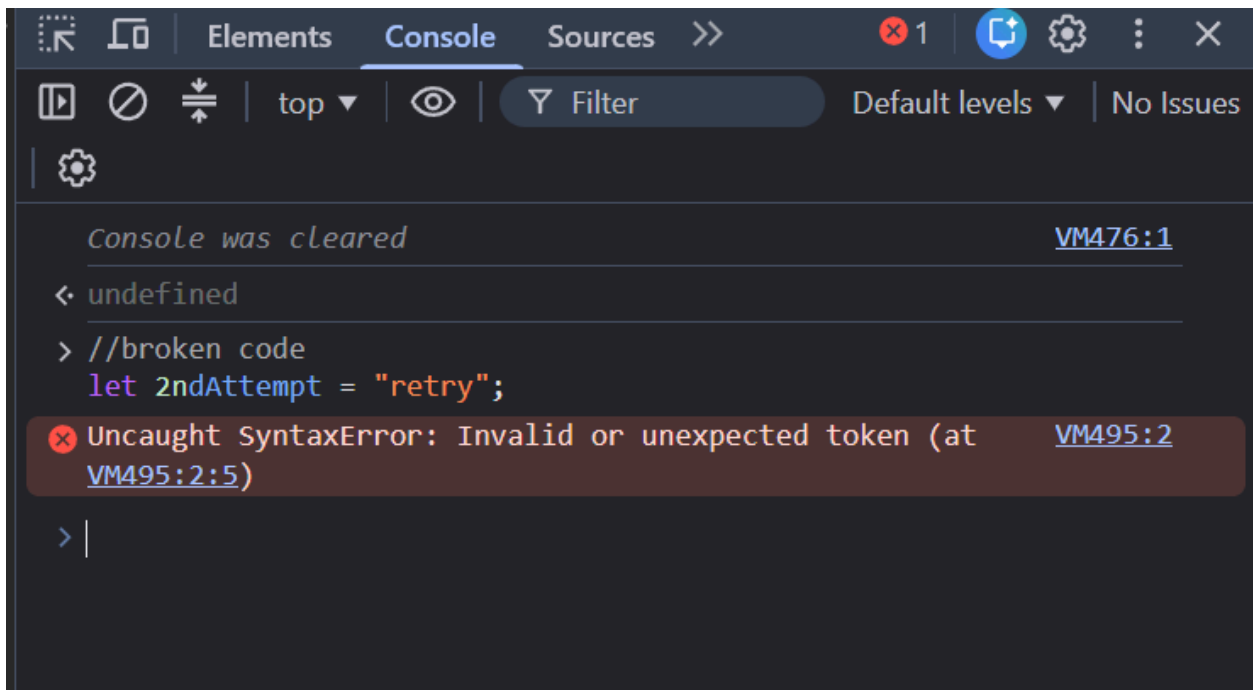
1. The + operator is incomplete. JavaScript expects another value after +.
2. The + operator was incomplete because no value was provided after it. I fixed it by adding 5.
3. username is accessed **before its declaration is executed**.
4. With `let`, the variable is in the **Temporal Dead Zone (TDZ)** until the declaration is reached.

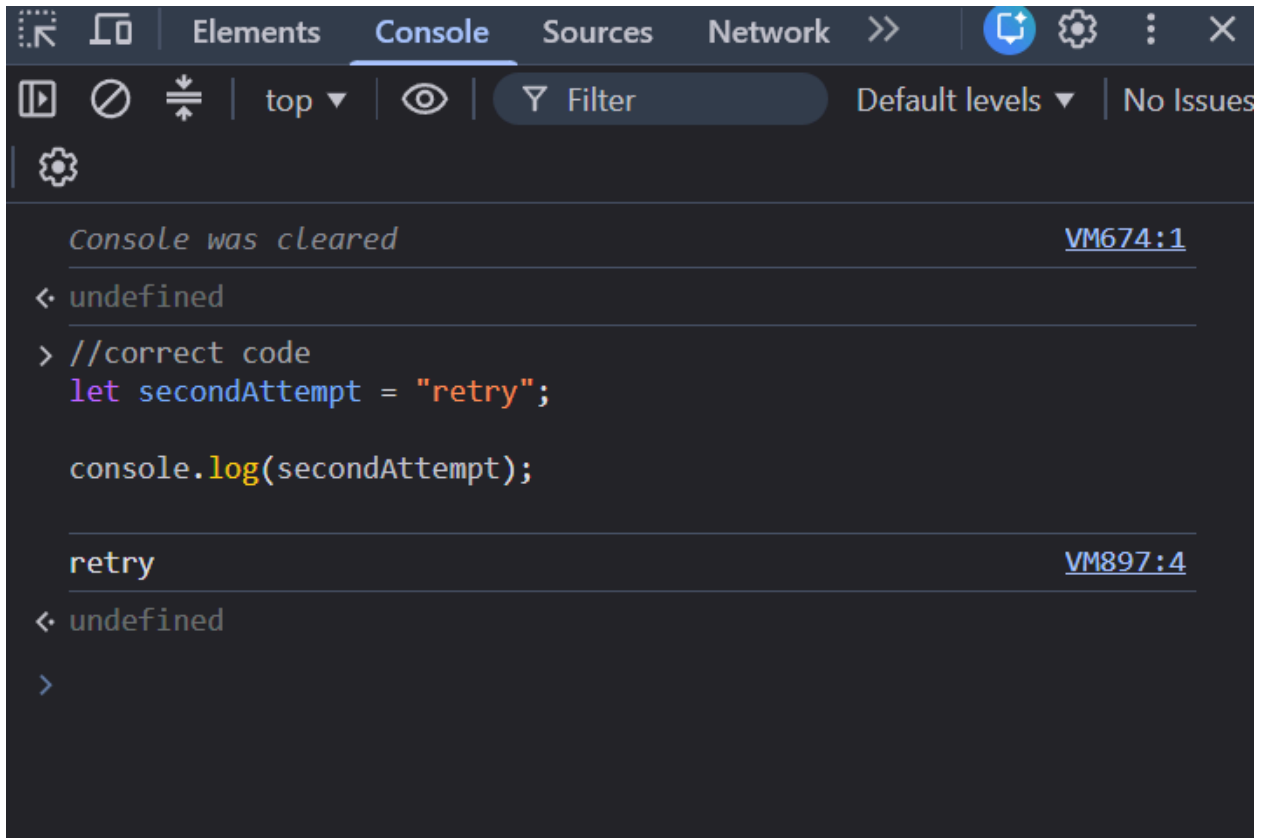


5.

The variable was used before it was initialized. I fixed the error by declaring and assigning username before using it.

Snippet -4:





Variable names cannot begin with a number. I changed 2ndAttempt to secondAttempt, which follows JavaScript's variable naming rules.

## Part 1 — Introduction

- Created index.html and script.js.
- Connected JS using `<script src="./script.js"></script>`.
- Output: Hello from script.js!

## Part 2 — Variables

- `let` → value can be reassigned.
- `const` → cannot be reassigned.
- `var` → function-scoped; `let` → block-scoped.
- `const` reassignment → `TypeError`.

## Part 3 — Data Types

- 7 primitives: **String, Number, Boolean, Undefined, Null, BigInt, Symbol**.
- `typeof null` → "object" due to historical behavior.
- Added Python to subjects and `isPassing`: `true` to object.

## Part 4 — Operators

- Arithmetic: +, -, \*, /, %
- == → type conversion allowed.
- === → value + type must match.
- Logical: &&, ||, !
- Ternary: condition ? true : false

## Part 5 — Grade Calculator

Total: 240, Average: 80.00, Grade: B

Average type: number

Scholarship Eligible: false

## Part 6 — Debugging

- Incomplete + → **SyntaxError**
- Reassigning const → **TypeError**
- Using let before initialization → **ReferenceError**
- Variable starting with number → **SyntaxError**

### Short paragraph:

The most confusing part of today's lab was understanding the difference between **let**, **const**, **var**, and their scope. Running the programs and checking the errors in the browser Console helped me understand the concepts better. Debugging the code made it easier to identify and fix common JavaScript errors.

== vs ===, null and object and some syntax errors also got confusing but solving the above problems helped me to clear the concept.